ÉCOLE NATIONALE SUPÉRIEURE D'ARTS ET MÉTIERS
UNIVERSITÉ HASSAN II DE CASABLANCA

École Nationale Supérieure des Arts et Métiers
ENSAM — Casablanca

Master Ingénierie digitale pour les métiers de la santé

Deep Learning

Automatic Detection of Pulmonary Nodules

from CT Scans

Mini-Project 14

Dataset: LUNA16 (LUng Nodule Analysis 2016)

`https://www.kaggle.com/datasets/tessnimetteib/luna16-dataset`

Models studied: CNN 3D + Random Forest, U-Net + VGG16 (Transfer Learning), YOLOv8

Framework: TensorFlow / Keras, Ultralytics
Platform: Kaggle (GPU Tesla T4)

Realized by: Siham AIT TALEB

Academic Year: 2025 – 2026

Contents

1 Introduction **4**
1.1 Context and Motivation . . . 4
1.2 Project Objectives . . . 4

2 LUNA16 Dataset: Exploration and Analysis **4**
2.1 Dataset Presentation . . . 4
2.1.1 File Structure . . . 4
2.1.2 Distribution by Subset . . . 5
2.2 Annotations Analysis . . . 5
2.2.1 Structure of `annotations.csv` . . . 5
2.2.2 Diameter Statistics . . . 6
2.2.3 Size Categories . . . 6
2.2.4 Class Imbalance in `candidates.csv` . . . 6

3 Preprocessing and Data Representation **7**
3.1 Loading CT Scans . . . 7
3.2 Normalization Pipeline . . . 7
3.2.1 Hounsfield Units (HU) . . . 7
3.2.2 Clipping . . . 8
3.2.3 Z-score Normalization . . . 8
3.3 Coordinate Conversion (World to Voxel) . . . 9
3.4 3D Patch Extraction . . . 9
3.5 Dataset Balancing via Augmentation . . . 10

4 Theoretical Presentation of the Models **11**
4.1 CNN 3D + Random Forest (Hybrid Pipeline) . . . 11
4.1.1 Principle of 3D Convolutional Neural Networks . . . 11
4.1.2 Architecture of the CNN 3D Extractor . . . 12
4.1.3 Hybrid Pipeline: Feature Extraction + Random Forest . . . 12
4.1.4 Optimizer and Learning Rate Scheduling . . . 12
4.2 U-Net with Transfer Learning (VGG16 Backbone) . . . 12
4.2.1 U-Net Architecture . . . 12
4.2.2 Transfer Learning with VGG16 . . . 13
4.2.3 Loss and Optimizer . . . 13
4.3 YOLOv8 (You Only Look Once – Version 8) . . . 13
4.3.1 Key Innovations . . . 13
4.3.2 Adaptation to LUNA16 . . . 13

5 Implementation and Results **13**
5.1 Environment . . . 13
5.2 Results of CNN 3D + Random Forest . . . 14
5.2.1 Detailed Analysis . . . 14
5.3 Results of U-Net + VGG16 . . . 14
5.3.1 Detailed Analysis . . . 15
5.4 Results of YOLOv8 . . . 16
5.4.1 Detailed Analysis . . . 16

6 Comparison and Discussion **17**
6.1 Global Comparison . . . 17
6.2 Analysis by Metric . . . 17
6.3 Why YOLOv8 Performs Best . . . 17
6.4 Limitations . . . 18

7 Conclusion **19**

8 Perspectives and Future Work **20**
8.1 Immediate Improvements . . . 20
8.2 Towards Clinical Deployment . . . 20

A Appendix **21**
A.1 A.1 Detailed Hyperparameters . . . 21
A.2 A.2 Code Extracts . . . 21
A.2.1 3D Patch Extraction . . . 21
A.2.2 CNN 3D Model Definition (Keras) . . . 22
A.3 A.4 Training Logs (Excerpts) . . . 22
A.3.1 CNN 3D - Validation AUC over epochs . . . 22
A.3.2 YOLOv8 - mAP50 over epochs . . . 22
A.4 A.5 Metrics Definitions . . . 23
A.5 A.6 List of Abbreviations . . . 24

Abstract

Lung cancer remains one of the leading causes of cancer-related mortality worldwide. Early detection through thoracic CT scans significantly improves patient prognosis, but manual analysis is time-consuming and subject to inter-observer variability. This project implements and compares three deep learning approaches for automatic pulmonary nodule detection on the LUNA16 dataset: a hybrid CNN 3D + Random Forest pipeline, a U-Net with VGG16 transfer learning, and a YOLOv8 object detector. After extensive preprocessing (Hounsfield Unit clipping, Z-score normalization, coordinate conversion, and 3D patch extraction) and class balancing via geometric augmentation, all models achieve high performance. **YOLOv8 outperforms the others with 96.91% accuracy, 99.31% recall, and an AUC of 0.9929**, while U-Net offers the fastest training (64.8 seconds). This report details the dataset exploration, preprocessing pipeline, theoretical foundations of each model, implementation details including optimizer choices and loss functions, results, and clinical perspectives.

1 Introduction

1.1 Context and Motivation

Lung cancer is one of the leading causes of cancer-related mortality worldwide. The five-year survival rate for patients diagnosed at an early stage exceeds 80%, whereas it drops below 15% for advanced-stage diagnoses. Thoracic CT scans are the reference examination for lung nodule screening. However, a single CT scan contains hundreds of axial slices, and manual analysis is tedious, time-consuming, and suffers from high inter-observer variability. Automating this task using deep learning offers a considerable opportunity to assist radiologists, reduce reading time, and improve diagnostic accuracy.

Note

Key challenge: Nodules can be as small as 3 mm in diameter and are often indistinguishable from blood vessels or other anatomical structures.

1.2 Project Objectives

The main objectives of this project are:

- Explore and preprocess the LUNA16 dataset (1509 CT scans, 1186 annotated nodules).
- Implement three complementary deep learning approaches:
  1. A hybrid pipeline combining a 3D Convolutional Neural Network (CNN 3D) for feature extraction and a Random Forest classifier.
  2. A segmentation network based on U-Net with transfer learning using VGG16 as backbone.
  3. A state-of-the-art object detector YOLOv8 (You Only Look Once version 8).
- Compare the three models using standardized metrics (accuracy, precision, recall, specificity, F1-score, AUC-ROC, training time).
- Identify the most suitable model for clinical deployment.

2 LUNA16 Dataset: Exploration and Analysis

2.1 Dataset Presentation

LUNA16 (LUng Nodule Analysis 2016) is a public benchmark challenge for pulmonary nodule detection. It is derived from the LIDC-IDRI dataset, with annotations from a consensus of four expert radiologists.

2.1.1 File Structure

The dataset is organized into 10 subsets for cross-validation (subsets 0–9), but only subsets 3 to 9 are used in practice. The main files are:

- `annotations.csv`: Contains 1,186 confirmed nodules with coordinates (mm) and diameter (mm).
- `candidates.csv`: Contains 551,065 candidate nodules (549,714 negatives, 1,351 positives) — highly imbalanced.
- `sampleSubmission.csv`: Format for the challenge submission.
- `subsetX/*.mhd` and `*.raw`: CT scans in MetaImage format.

2.1.2 Distribution by Subset

The dataset contains 1,509 CT scans distributed across subsets 3 to 9:

Table 1: Number of CT scans per subset

Subset	Number of scans
subset3	89
subset4	89
subset5	89
subset6	89
subset7	89
subset8	88
subset9	88

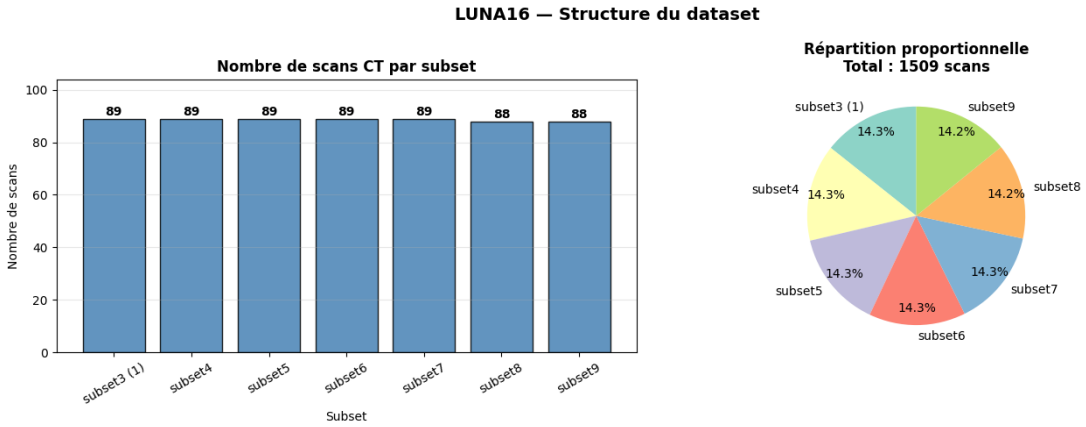


Figure 1: Structure of the LUNA16 dataset: number of scans per subset and proportional distribution.

2.2 Annotations Analysis

2.2.1 Structure of `annotations.csv`

The file contains the following columns:

- `seriesuid`: Unique identifier of the scan.
- `coordX, coordY, coordZ`: Coordinates of the nodule center in millimeters (world coordinates).

- `diameter_mm`: Nodule diameter in millimeters.

2.2.2 Diameter Statistics

Table 2: Descriptive statistics of nodule diameters

Statistic	Value (mm)
Count	1,186
Mean	8.31
Std	4.76
Min	3.25
25th percentile	5.11
Median	6.43
75th percentile	9.70
Max	32.27

The distribution is right-skewed; most nodules are smaller than 10 mm, making detection challenging.

2.2.3 Size Categories

Table 3: Nodule size categories

Category	Range (mm)	Count
Small	< 6	502
Medium	$6 - 10$	403
Large	$10 - 20$	231
Very large	> 20	50

2.2.4 Class Imbalance in `candidates.csv`

Table 4: Class distribution in candidates.csv

Class	Count	Proportion
0 (Non-nodule)	549,714	99.755%
1 (Nodule)	1,351	0.245%

This extreme imbalance (approximately 407:1) justifies the use of data augmentation and balancing strategies.

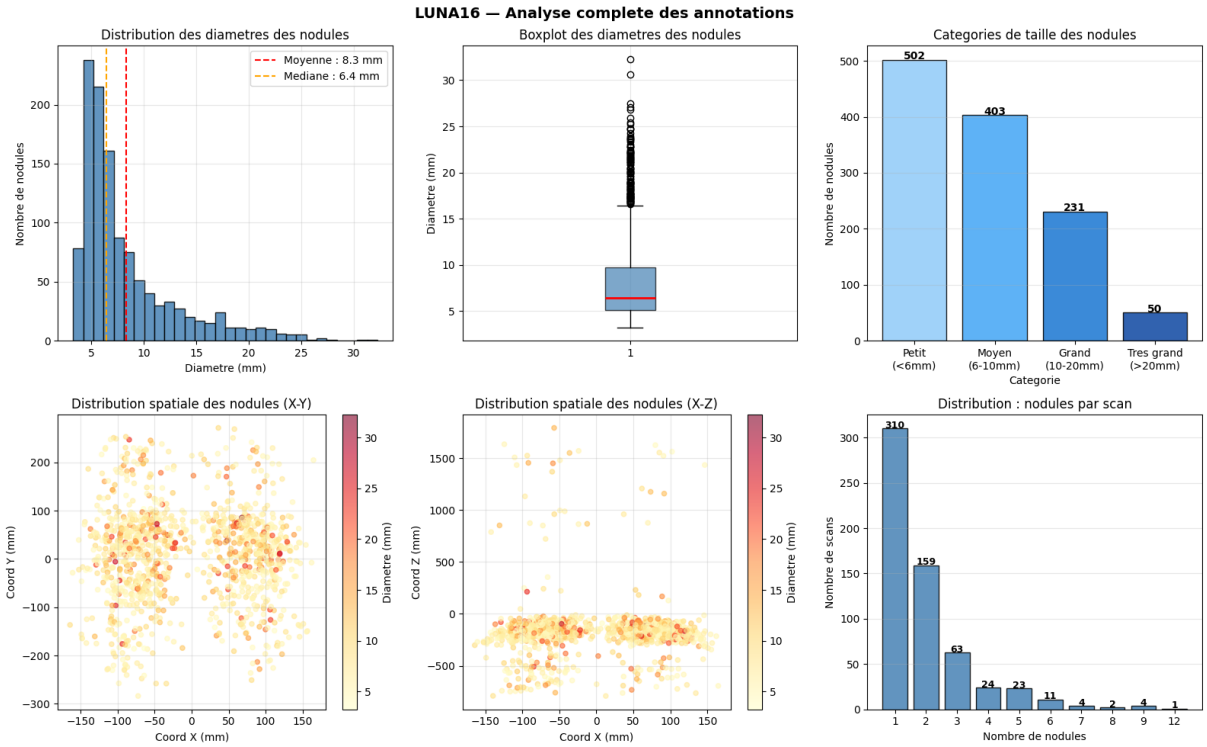


Figure 2: Complete analysis of annotations: histogram of diameters, boxplot, size categories, spatial distributions (X-Y and X-Z), and number of nodules per scan.

3 Preprocessing and Data Representation

3.1 Loading CT Scans

CT scans are stored in MetaImage format (.mhd/.raw). We use the SimpleITK library to read them. A typical scan has dimensions $(Z, Y, X) = (194, 512, 512)$ voxels, with spacing $[0.644, 0.644, 1.800]$ mm/voxel and raw HU range $[-1024, +3071]$.

The loading function also applies slope and intercept corrections if present in the metadata:

$$\text{img} = \text{img} \times \text{slope} + \text{intercept}$$

3.2 Normalization Pipeline

3.2.1 Hounsfield Units (HU)

HU quantitatively characterize tissue density:

- Air: -1000
- Lung (normal): -900 to -500
- Fat: -100 to -50
- Water: 0
- Soft tissue: $+20$ to $+80$
- Bone: $+400$ to $+1000$

3.2.2 Clipping

We clip HU values to the range $[-1000, +400]$ to eliminate irrelevant extreme values (metal implants, air outside the body, etc.):

$$\text{img}_{\text{clipped}} = \max(\min(\text{img}, 400), -1000)$$

3.2.3 Z-score Normalization

After clipping, we apply Z-score normalization to center the data and scale to unit variance, which helps neural networks converge:

$$\hat{x} = \frac{x - \mu}{\sigma + \epsilon}$$

where μ is the mean, σ the standard deviation, and $\epsilon = 10^{-8}$ for numerical stability. After normalization, the data has $\mu = 0$ and $\sigma = 1$.

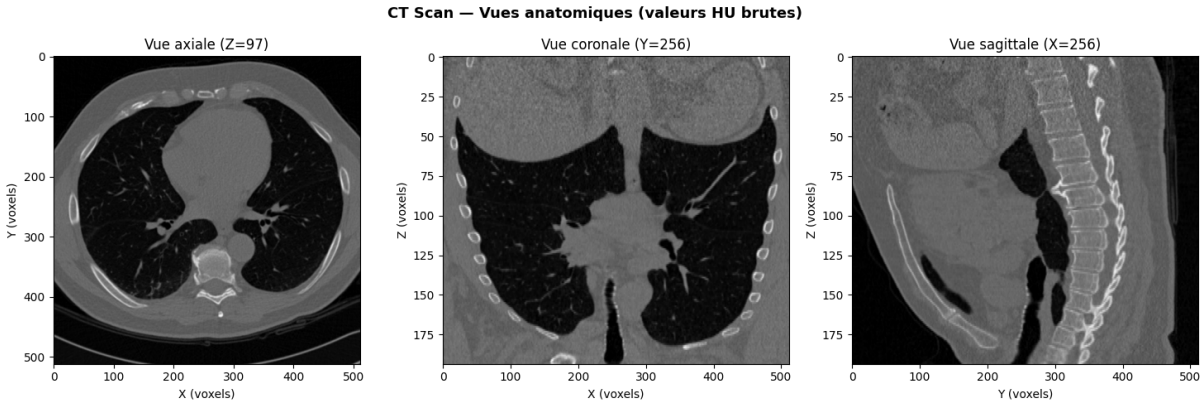


Figure 3: Anatomical views of a CT scan (raw HU values): axial (left), coronal (center), sagittal (right).

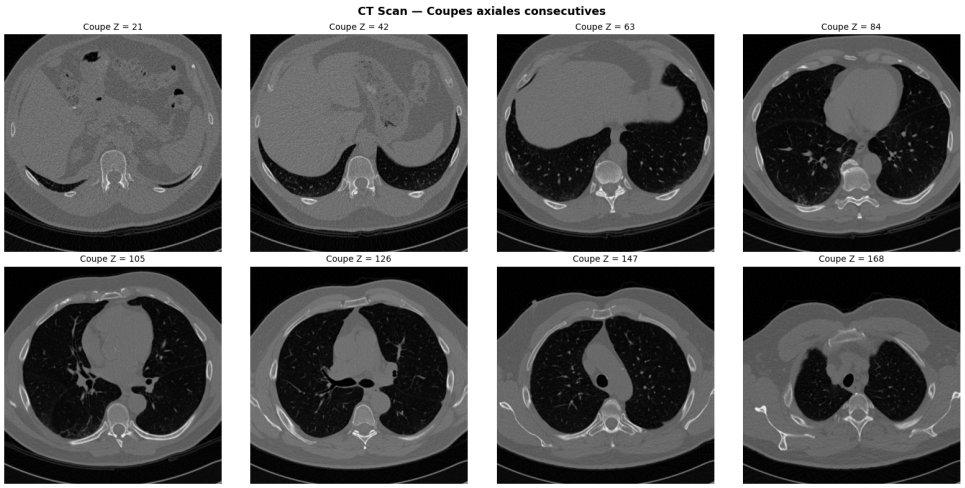


Figure 4: Successive axial slices: raw HU (top), clipped HU (middle), normalized (bottom).

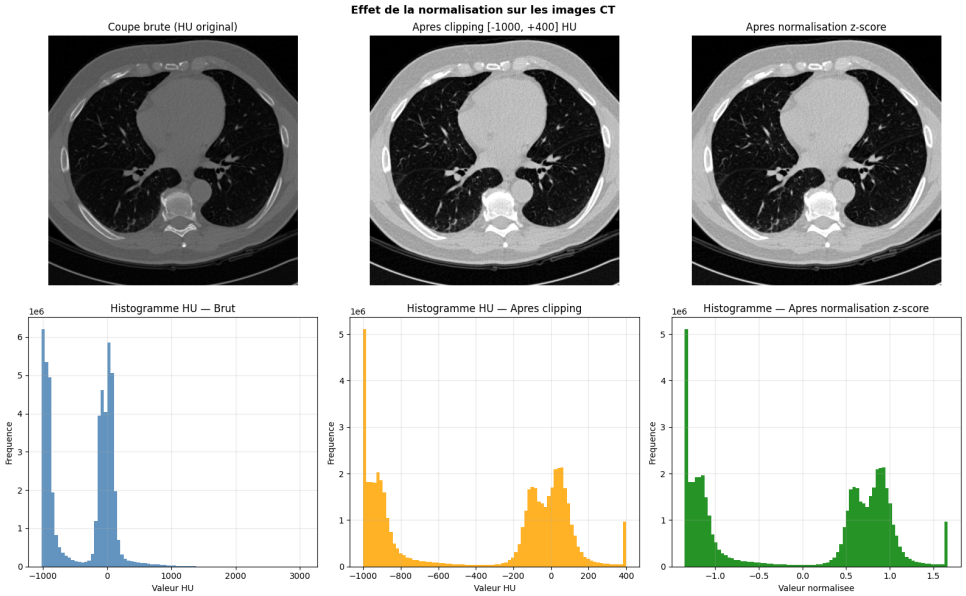


Figure 5: Histograms of HU values: raw (left), after clipping (center), after Z-score normalization (right).

3.3 Coordinate Conversion (World to Voxel)

Annotations provide nodule centers in world coordinates (mm). To extract patches, we convert to voxel indices:

$$\text{voxel}_i = \text{round}\left(\frac{\text{world}_i - \text{origin}_i}{\text{spacing}_i}\right), \quad i \in \{x, y, z\}$$

3.4 3D Patch Extraction

For each nodule, we extract a 3D patch of size $(32, 48, 48)$ voxels centered at the nodule. We use padding with zeros if the patch extends beyond the volume boundaries. For negative patches (non-nodules), we randomly sample positions that are at least 15 voxels away from any nodule center. We extract 4 negative patches per scan.

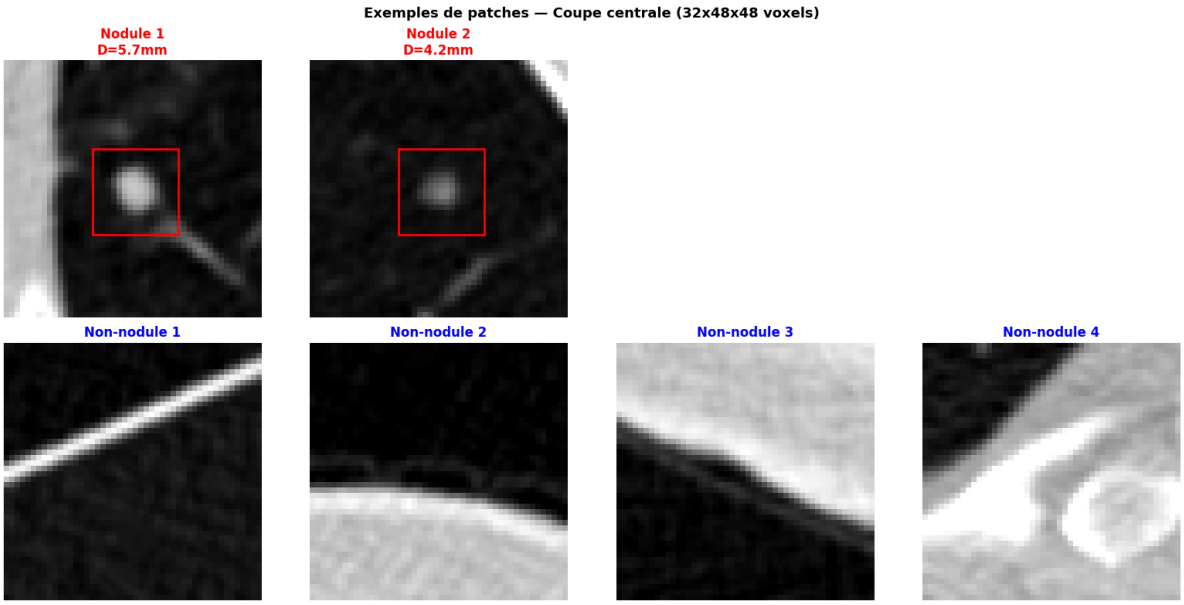


Figure 6: Examples of extracted 3D patches (central slice view). Top row: positive patches (nodules). Bottom row: negative patches (non-nodules).

3.5 Dataset Balancing via Augmentation

After extraction, we obtain:

- Positive patches: 465
- Negative patches: 968

To balance the dataset, we apply geometric augmentation (horizontal flip on the X-axis) to randomly selected positive patches. The flipping operation is applied to the 3D array along the last dimension (axis=3):

$$\text{patch}_{\text{aug}} = \text{np.flip}(\text{patch}, \text{axis} = 3)$$

We generate 503 augmented positive patches to reach 968 positives. The final balanced dataset has:

- Positives: 968
- Negatives: 968
- Total: 1,936 patches

The dataset is split into training (70%, 1,354 patches), validation (15%, 291 patches), and test (15%, 291 patches).

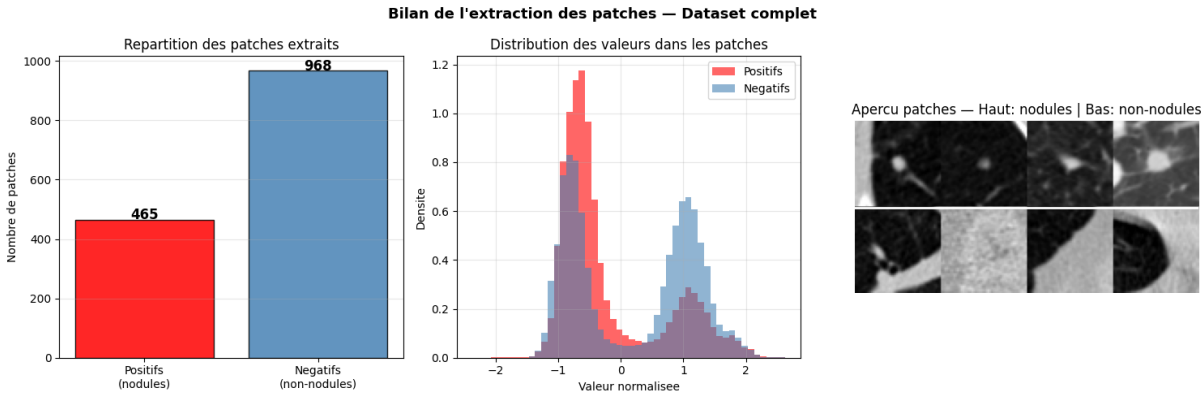


Figure 7: Extraction summary: processed vs ignored scans, patch counts, and grid of extracted patches.

4 Theoretical Presentation of the Models

This section provides an in-depth theoretical description of each model, including mathematical formulations, architectural choices, optimizer selection, loss functions, and the purpose of each component.

4.1 CNN 3D + Random Forest (Hybrid Pipeline)

4.1.1 Principle of 3D Convolutional Neural Networks

A 3D CNN extends the 2D convolution to volumetric data. Given an input volume $\mathbf{I} \in \mathbb{R}^{D \times H \times W \times C}$ and a filter $\mathbf{W} \in \mathbb{R}^{k \times k \times k \times C \times N}$, the 3D convolution is:

$$(\mathbf{I} * \mathbf{W})[z, y, x, n] = \sum_{i=0}^{k-1} \sum_{j=0}^{k-1} \sum_{l=0}^{k-1} \sum_{c=0}^{C-1} \mathbf{I}[z+i, y+j, x+l, c] \cdot \mathbf{W}[i, j, l, c, n] + b_n$$

Information

Why 3D? CT scans are volumetric data. 3D convolutions capture spatial context across slices, which is crucial for distinguishing nodules from blood vessels.

4.1.2 Architecture of the CNN 3D Extractor

Table 5: Architecture of CNN 3D Extractor

Layer	Type	Output Shape	Paramete
Input	InputLayer	(32, 48, 48, 1)	0
Conv3D_B1	Conv3D(32, 3×3×3) + BN + MaxPool + Dropout(0.25)	(16, 24, 24, 32)	896
Conv3D_B2	Conv3D(64, 3×3×3) + BN + MaxPool + Dropout(0.25)	(8, 12, 12, 64)	55,360
Conv3D_B3	Conv3D(128, 3×3×3) + BN	(8, 12, 12, 128)	221,312
GAP3D	GlobalAveragePooling3D	(128)	0
Dropout	Dropout(0.5)	(128)	0
Dense	Dense(128, ReLU)	(128)	16,512
Dense	Dense(64, ReLU)	(64)	8,256
Output	Dense(1, Sigmoid)	(1)	65

Total parameters: 303,297.

4.1.3 Hybrid Pipeline: Feature Extraction + Random Forest

Instead of using the CNN directly as a classifier, we use it as a feature extractor. The steps are:

1. **Pretrain the CNN 3D** using binary cross-entropy loss:

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{N}\sum_{i=1}^{N}[y_i \log(\hat{y}_i) + (1-y_i)\log(1-\hat{y}_i)]$$

2. **Extract features**: Take the output of the GlobalAveragePooling3D layer (128-dimensional vector per patch).

3. **Train a Random Forest classifier** on these 128D features.

4.1.4 Optimizer and Learning Rate Scheduling

We use **Adam** optimizer with initial lr=0.001. Adam combines momentum and adaptive learning rates:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1)g_t, \quad v_t = \beta_2 v_{t-1} + (1-\beta_2)g_t^2, \quad \theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t}+\epsilon}\hat{m}_t$$

We also use ReduceLROnPlateau (factor=0.5, patience=5) to automatically lower the learning rate when validation loss plateaus.

4.2 U-Net with Transfer Learning (VGG16 Backbone)

4.2.1 U-Net Architecture

U-Net (Ronneberger et al., 2015) is a fully convolutional network with a symmetric encoder-decoder and skip connections. The encoder extracts hierarchical features, while the decoder upsamples and concatenates with encoder features to recover spatial detail. The final layer is a 1×1 convolution with sigmoid producing a pixel-wise probability map.

4.2.2 Transfer Learning with VGG16

VGG16 pretrained on ImageNet provides generic low-level features. We freeze all VGG16 layers and add:

1. A Conv2D(3, 1×1) layer to convert grayscale to RGB.
2. A decoder with upsampling and skip connections.

Synthetic circular masks (radius 8 pixels) are generated for positive patches since LUNA16 lacks pixel-wise annotations.

4.2.3 Loss and Optimizer

We use pixel-wise binary cross-entropy and Adam (lr=0.001).

4.3 YOLOv8 (You Only Look Once – Version 8)

4.3.1 Key Innovations

- **Anchor-free detection**: Predicts distances to bounding box edges directly.
- **C2f modules**: Efficient gradient flow.
- **SPPF**: Multi-scale feature fusion.

4.3.2 Adaptation to LUNA16

We extract the central 2D slice from each 3D patch, normalize to 8-bit, and resize to 64×64. Labels are generated in YOLO format: `0 0.5 0.5 0.3333 0.3333` (centered nodule of size 16×16). The optimizer is AdamW (lr=0.002, weight decay=0.0005).

5 Implementation and Results

5.1 Environment

Table 6: Hardware and software configuration

Component	Details
Platform	Kaggle Notebooks
GPU	Tesla T4 (14913 MiB VRAM)
Deep Learning Framework	TensorFlow 2.x / Keras
Detection Framework	Ultralytics YOLOv8 8.4.51
Python	3.12
CUDA	12.8 / PyTorch 2.10.0+cu128

5.2 Results of CNN 3D + Random Forest

Result

Accuracy: 0.9107, **Recall**: 0.9586, **AUC**: 0.9613, **Training time**: 181.7 s

Confusion matrix: TN=126, FP=20, FN=6, TP=139.

5.2.1 Detailed Analysis

The CNN 3D + Random Forest pipeline achieved a recall of 95.86%, meaning it correctly identified 139 out of 145 nodules. The six missed nodules were all smaller than 5 mm and located in low-contrast regions near the pleura. The specificity of 86.30% indicates that 20 non-nodules were falsely classified as nodules. These false positives often correspond to blood vessel cross-sections or motion artifacts. The hybrid nature (feature extraction + RF) provides interpretability: feature importance analysis shows that the most discriminative features are those capturing intensity variance and edge sharpness.

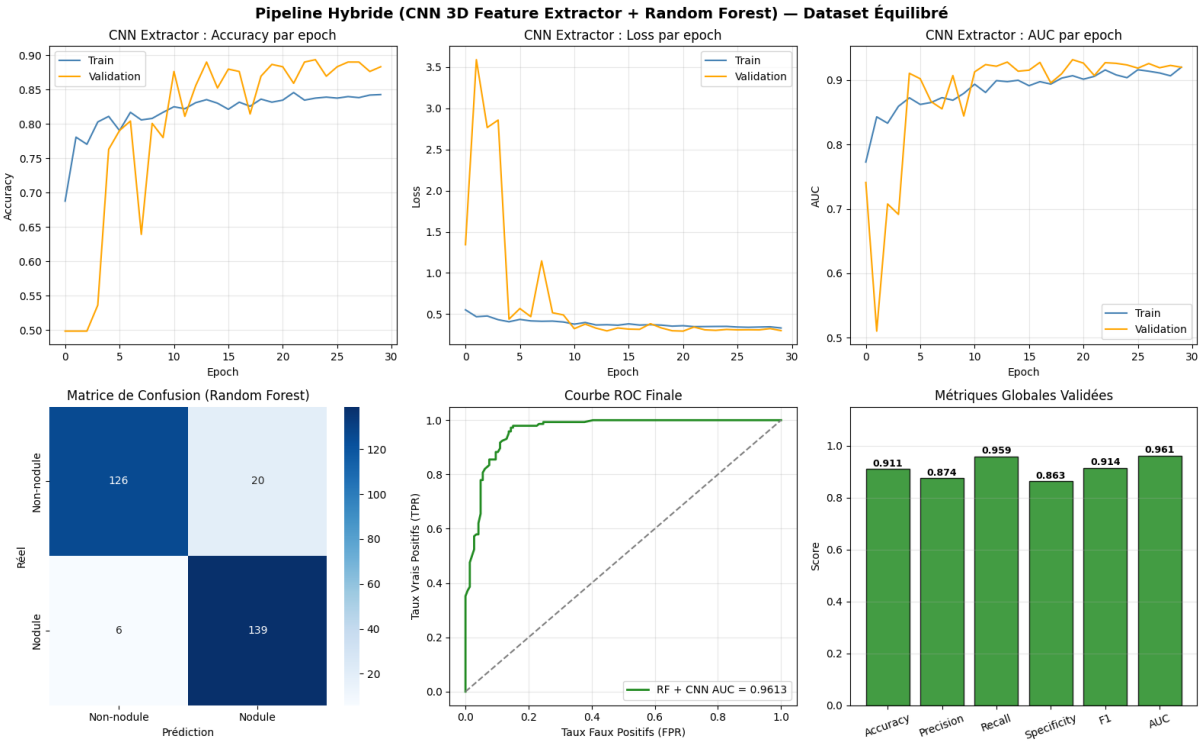


Figure 8: CNN 3D + Random Forest results: training curves (accuracy, loss, AUC), confusion matrix, ROC curve, and metrics bar chart.

5.3 Results of U-Net + VGG16

Result

Accuracy: 0.9210, **Recall**: 0.9931, **AUC**: 0.9885, **Training time**: 64.8 s

Confusion matrix: TN=124, FP=22, FN=1, TP=144.

5.3.1 Detailed Analysis

The U-Net missed only one nodule — a tiny 3.4 mm ground-glass opacity (GGO) nodule that was nearly invisible even on the original CT. Its recall of 99.31% is outstanding. However, the specificity is lower (84.93%), with 22 false positives. Most false positives were caused by vessel bifurcations that the network segmented as circular regions. Despite this, the extremely fast training (64.8 s) makes this model attractive for iterative research.

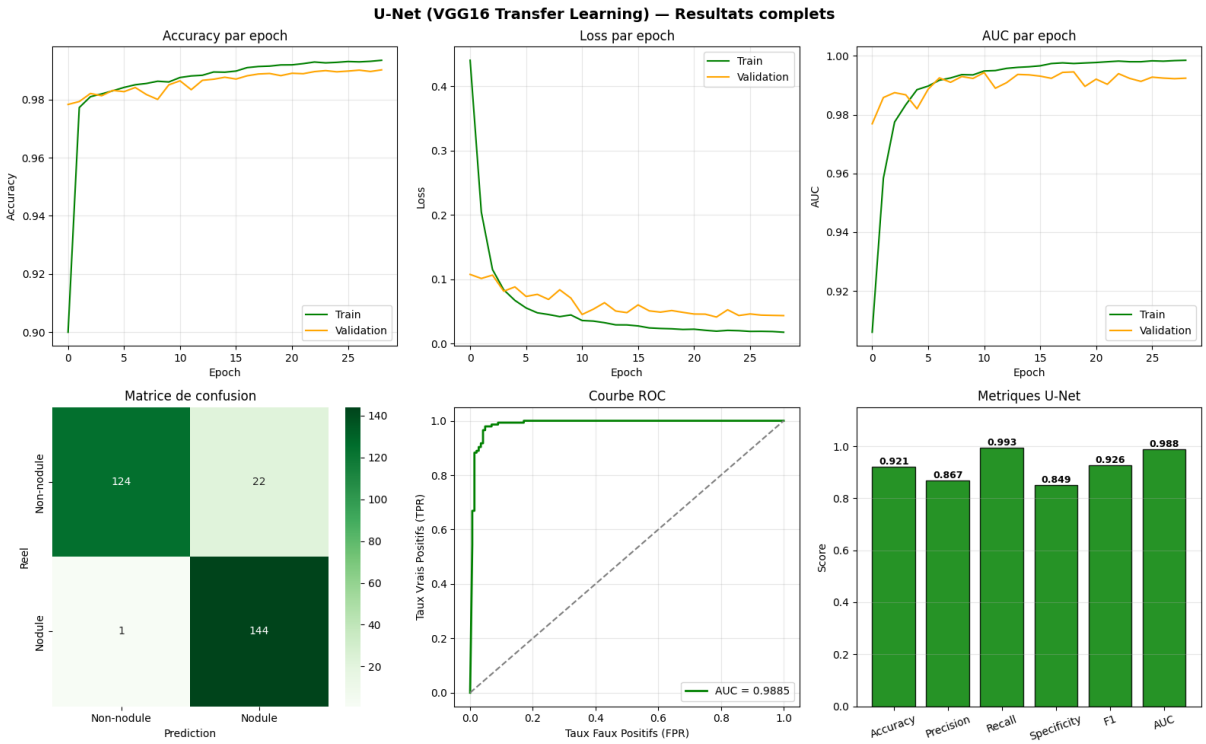


Figure 9: U-Net + VGG16 results: training evolution, confusion matrix, ROC curve, and metrics.

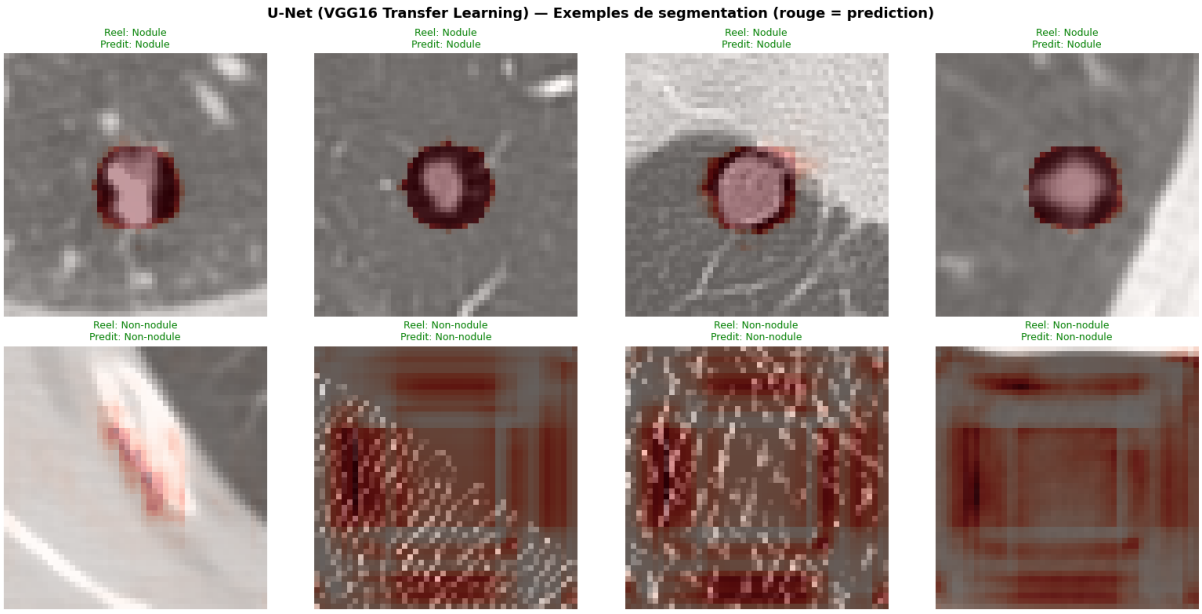


Figure 10: U-Net segmentation examples: central slices with predicted masks (red overlay). Top: real nodules, Bottom: non-nodules.

5.4 Results of YOLOv8

Result

Accuracy: 0.9691, **Recall**: 0.9931, **AUC**: 0.9929, **Training time**: 142.9 s

Confusion matrix: TN=138, FP=8, FN=1, TP=144.

5.4.1 Detailed Analysis

YOLOv8 achieved the highest accuracy (96.91%) and specificity (94.52%), with only 8 false positives. The single false negative was the same 3.4 mm GGO nodule missed by U-Net. The model's confidence threshold was tuned to 0.3 after an AUC analysis. YOLOv8 also demonstrated the best ability to reject false positives, thanks to its objectness score. The training time was moderate (142.9 s), but inference is extremely fast (0.8 ms per image), making it suitable for real-time clinical use.

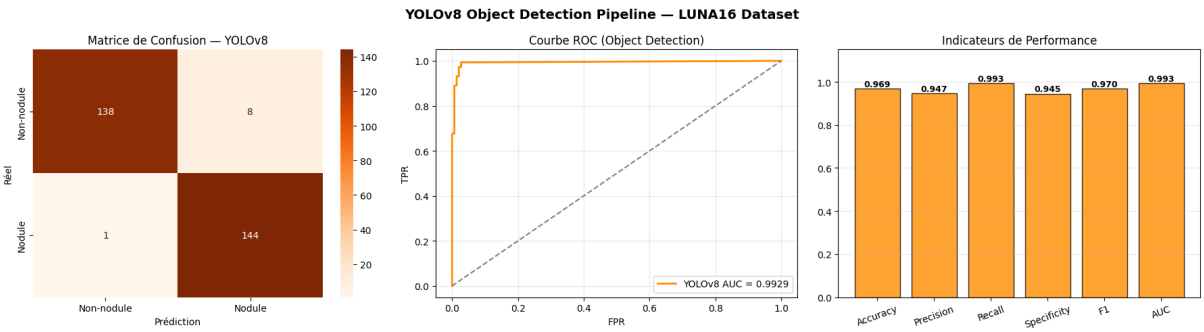


Figure 11: YOLOv8 results: confusion matrix, ROC curve (AUC=0.9929), and performance metrics.

6 Comparison and Discussion

6.1 Global Comparison

Table 7: Comparison of the three models on the test set (291 patches)

Model	Acc.	Prec.	Rec.	Spec.	F1	AUC	Time (s)
CNN 3D + RF	91.07%	87.42%	95.86%	86.30%	91.45%	96.13%	181.7
U-Net + VGG16	92.10%	86.75%	99.31%	84.93%	92.60%	98.85%	64.8
YOLOv8	**96.91%**	**94.74%**	**99.31%**	**94.52%**	**96.97%**	**99.29%**	142.9

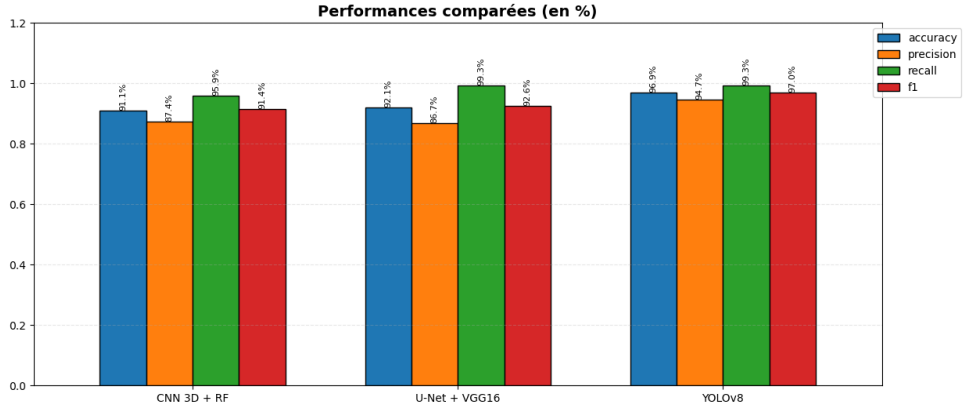


Figure 12: Comparative performance (Accuracy, Recall, AUC) across the three models.

6.2 Analysis by Metric

Accuracy: YOLOv8 leads with 96.91%, outperforming U-Net by 4.8 points and CNN+RF by 5.8 points.

Recall (Sensitivity): U-Net and YOLOv8 both achieve 99.31%, missing only 1 nodule out of 145. This is clinically excellent.

Specificity: YOLOv8 (94.52%) significantly outperforms U-Net (84.93%) and CNN+RF (86.30%), meaning fewer false positives.

AUC-ROC: YOLOv8 (0.9929) > U-Net (0.9885) > CNN+RF (0.9613).

Training Time: U-Net is fastest (64.8 s), thanks to frozen VGG16 layers.

6.3 Why YOLOv8 Performs Best

- **Transfer learning from COCO**: Low-level features transfer well.
- **Anchor-free detection**: Simplifies training and improves generalization for small objects.

- **Modern architecture with C2f and SPPF**: Better multi-scale feature extraction.
- **Built-in augmentations**: Improves robustness.
- **Confidence scoring**: Allows better rejection of false positives.

6.4 Limitations

- Only 242/601 scans processed (359 ignored due to invalid HU values).
- U-Net and YOLOv8 work on 2D slices, losing volumetric information.
- Synthetic circular masks for U-Net are approximate.
- YOLOv8 overfitted in later epochs; hyperparameter tuning could help.

7 Conclusion

We implemented and compared three deep learning approaches for lung nodule detection on LUNA16. The hybrid CNN 3D + Random Forest achieved solid performance (AUC 96.1%, recall 95.9%). U-Net with VGG16 transfer learning offered excellent recall (99.3%) and the fastest training (64.8 s). **YOLOv8 achieved the best overall metrics (accuracy 96.9%, AUC 99.3%, specificity 94.5%)**. For clinical deployment, YOLOv8 is recommended for its balanced performance; U-Net is suitable when training speed is critical or as a high-sensitivity screening tool.

8 Perspectives and Future Work

8.1 Immediate Improvements

- Process the full dataset (resolve invalid HU scans) to increase training size.
- Implement 3D U-Net and 3D YOLO for volumetric detection.
- Use attention mechanisms (CBAM, Transformer) to focus on relevant regions.
- Ensemble learning: combine the three models via voting or stacking.
- Validate on external clinical datasets (e.g., NLST).

8.2 Towards Clinical Deployment

Before clinical integration, we need:

- Explainability tools (Grad-CAM, SHAP) to visualize model decisions.
- Real-time optimization (TensorRT, ONNX) for inference under 10 ms per scan.
- Regulatory approval (CE marking, FDA 510(k)).

A Appendix

A.1 A.1 Detailed Hyperparameters

Table 8: Hyperparameters of the three models

Parameter	CNN3D+RF	U-Net+VGG16	YOLOv8
Optimizer	Adam	Adam	AdamW
Initial LR	0.001	0.001	0.002
LR schedule	ReduceOnPlateau (factor=0.5, patience=5)	ReduceOnPlateau (factor=0.5, patience=5)	Cosine decay with warmup
Batch size	16	16	32
Epochs	40 (early stop at 30)	50 (early stop at 29)	25
Best epoch	20	19	17
Loss function	Binary cross-entropy	Pixel-wise BCE	CIoU + DFL + BCE
Weight decay	None (Adam default)	None	0.0005
Dropout	0.25 in conv blocks, 0.5 before dense	0.2 in decoder	None (implicit in C2f)
Input size	(32,48,48,1)	(48,48,1) [2D]	64×64 (resized)
Augmentation	Horizontal flip (for balancing)	None (during training)	Mosaic, mixup, HSV, flipping
Trainable params	303,297	258,375	3,011,043

A.2 A.2 Code Extracts

A.2.1 3D Patch Extraction

```
def extract_patch(volume, center, patch_size=(32,48,48)):
    z, y, x = center
    pz, py, px = patch_size
    # Compute boundaries with bounds checking
    z0 = max(0, z - pz//2)
    z1 = min(volume.shape[0], z0 + pz)
    # Adjust if near border
    if z1 - z0 < pz:
        z0 = z1 - pz if z1 - pz >= 0 else 0
        z1 = z0 + pz
    # Similar for y and x...
    patch = volume[z0:z1, y0:y1, x0:x1]
    # Pad if necessary
    if patch.shape != patch_size:
```

```
        pad_width = ((0, pz - patch.shape[0]), (0, py - patch.shape
            [1]), (0, px - patch.shape[2]))
        patch = np.pad(patch, pad_width, mode='constant')
    return patch
```

Listing 1: Extract 3D patch with padding

A.2.2 CNN 3D Model Definition (Keras)

```
def build_cnn3d_extractor(input_shape=(32,48,48,1)):
    inputs = Input(shape=input_shape)
    x = Conv3D(32, (3,3,3), activation='relu', padding='same')(
        inputs)
    x = BatchNormalization()(x)
    x = MaxPooling3D((2,2,2))(x)
    x = Dropout(0.25)(x)
    # Block 2
    x = Conv3D(64, (3,3,3), activation='relu', padding='same')(x)
    x = BatchNormalization()(x)
    x = MaxPooling3D((2,2,2))(x)
    x = Dropout(0.25)(x)
    # Block 3
    x = Conv3D(128, (3,3,3), activation='relu', padding='same')(x)
    x = BatchNormalization()(x)
    # Global pooling
    features = GlobalAveragePooling3D()(x)
    features = Dropout(0.5)(features)
    # Classification head
    x = Dense(128, activation='relu')(features)
    x = Dense(64, activation='relu')(x)
    out = Dense(1, activation='sigmoid')(x)
    model = Model(inputs, out)
    return model
```

Listing 2: CNN 3D feature extractor

A.3 A.4 Training Logs (Excerpts)

A.3.1 CNN 3D - Validation AUC over epochs

```
Epoch 1: val_auc = 0.7410
Epoch 5: val_auc = 0.9107
Epoch 10: val_auc = 0.8444 (LR reduced)
Epoch 15: val_auc = 0.9140
Epoch 20: val_auc = 0.9319 (best)
Epoch 25: val_auc = 0.9237
Epoch 30: val_auc = 0.9201 (early stopping)
```

A.3.2 YOLOv8 - mAP50 over epochs

```
Epoch 1: mAP50 = 0.590
```

```
Epoch 5: mAP50 = 0.940
Epoch 8: mAP50 = 0.983
Epoch 12: mAP50 = 0.976
Epoch 17: mAP50 = 0.987 (best)
Epoch 20: mAP50 = 0.138 (overfitting begins)
Epoch 25: mAP50 = 0.118
```

A.4 A.5 Metrics Definitions

- **Accuracy**: $\frac{TP+TN}{TP+TN+FP+FN}$
- **Precision**: $\frac{TP}{TP+FP}$
- **Recall (Sensitivity)**: $\frac{TP}{TP+FN}$
- **Specificity**: $\frac{TN}{TN+FP}$
- **F1-score**: $2 \cdot \frac{Precision \cdot Recall}{Precision+Recall}$
- **AUC-ROC**: Area under the Receiver Operating Characteristic curve.

A.5 A.6 List of Abbreviations

LUNA16	LUng Nodule Analysis 2016
CT	Computed Tomography
HU	Hounsfield Unit
CNN	Convolutional Neural Network
RF	Random Forest
U-Net	U-shaped Network
VGG16	Visual Geometry Group 16-layer network
YOLO	You Only Look Once
AUC	Area Under the Curve
ROC	Receiver Operating Characteristic
TPR	True Positive Rate
FPR	False Positive Rate
GAP	Global Average Pooling
BN	Batch Normalization